

Holocron — Capstone Problem Brief

Week 8 capstone handout. This is the problem your triad builds the capstone artifact set from. It gives you the business situation, the people involved, and the facts you need — it does **not** give you requirements. Deriving the personas, scope, acceptance criteria, and non-functional requirements is the capstone.

The situation

You are the Technical Product Manager assigned to **Holocron**, a proposed internal platform capability.

Across the enterprise, the text that appears in user interfaces — labels, buttons, error messages, help text, legal notices — is managed inconsistently. Much of it is hardcoded directly in application code. Some lives in scattered files owned by whichever team built the screen.

The practical result: **changing a sentence requires an engineer and a deployment.** A one-word correction to a legal disclaimer, a fix to a confusing error message, or a seasonal change in wording becomes a ticket, a code change, a review, and a release cycle. Work that should take minutes takes days or weeks.

Why the business cares

Three pressures are driving this now.

Delivery drag. Engineering capacity is consumed by text edits that carry no engineering value. Business and content teams cannot make timely changes, so copy stays wrong longer than anyone wants.

Compliance and governance risk. Some strings are compliance-sensitive — legal disclaimers, regulatory notices, privacy language. Today these can move through a release without a consistent legal review gate, and there is no centralized, durable record of who approved what and when. That is an audit exposure.

Global inconsistency. Ownership is decentralized and translation processes vary by team, which produces uneven quality and unpredictable operational risk.

The mandate

Leadership has asked for a **centralized, governed way to manage UI text** so that content owners can create, review, and publish text without hardcoding it or redeploying applications — while preserving traceability, clear ownership, and approval controls.

That sentence is the whole of the direction you have been given. **What it should actually do, for whom, in what order, and to what quality bar is your work.**

Facts you can build on

These are established. Use them as evidence — do not invent numbers beyond them, and flag anything you need that is missing.

Dimension	Fact
Countries of operation	150+
Consuming applications	20–30+
Languages / locales	30–40
Source language	en-US is the authoritative source; translations derive from it
Expected content volume	The platform is expected to hold on the order of 500,000 strings
Identity	Enterprise identity services already exist and are expected to cover authentication and role mapping
Hosting	Enterprise Kubernetes (AKS); provisioning runs through Infrastructure
Data handling	Enterprise encryption and data-classification standards apply, in transit and at rest
Translation vendors	Work through an exchange-file workflow; vendors do not get in-app access in this version

Who is involved

Your discovery and stakeholder work should account for these roles. Titles are real; their goals, frustrations, and influence are for you to establish.

Role	Relationship to Holocron
Content Owner	Creates and maintains the text; today blocked behind engineering
Translator / translation vendor	Produces locale content; works through file exchange
Engineer (consumer)	Integrates applications that render the text; wants to stop hardcoding
Legal / Compliance reviewer	Must approve compliance-sensitive text; needs durable audit evidence
Enterprise Infrastructure	Owns identity integration and environment provisioning
Engineering Architecture	Owns key/namespace governance and the delivery-mechanism decision
Product	Owns scope, priority, and the source-of-truth policy

Known dependencies

Real, already-identified, with owners. These belong in your delivery and stakeholder thinking.

Dependency	Owner	Risk if it slips
Enterprise identity integration	Enterprise Infrastructure	Cannot enforce scoped authorization
Translation partner + exchange format decision	Product + Stakeholders	Translation workflow blocked
Key schema and namespace governance	Architecture	Rework and migration risk
Delivery mechanism architecture decision	Engineering Architecture	Consumer integration blocked
Kubernetes namespace + database provisioning	Infrastructure	Environment readiness slips
Audit retention policy confirmation	Legal / Compliance	Governance non-compliance

Already ruled out for this version

The business has explicitly deferred these. They are useful raw material for your non-goals — but you still have to decide and defend your own scope line.

- AI-assisted translation automation
- Design-tool (Figma) integration for key assignment
- In-app access for external translation vendors
- Per-locale screenshot preview and layout simulation
- A/B testing and regional content variants
- Storing binary assets (images, screenshots) in the system
- Any public or unauthenticated access to text delivery

Open questions the business has not answered

These are genuinely undecided. A strong capstone does not paper over them — it names them, states an assumption, or routes them to the right owner.

- What the lifecycle states for source text and translations should finally be
 - How quickly a published change must reach consuming applications
 - What should happen to a previously approved string when someone rolls back
 - What peak demand the delivery mechanism must sustain
 - Whether review is optional or required in the publish workflow
 - How shared/reused terminology across products should be governed
-

What you produce

The Holocron problem is the input to the full Week-8 capstone artifact set — the same compressed chain you have built all course:

Discovery → PRD-light → TCD-light + TMD-light → AI Spec → SEP-light + DP-light → Friday presentation.

Everything in this brief is context. The requirements are yours to write.

The scoping reality

You have four working days. You cannot build a specification for every capability an enterprise string platform could have. Part of the assessment is **choosing a defensible slice** — the capabilities that make the first release genuinely useful — and being explicit about what you deliberately left for later, and why.